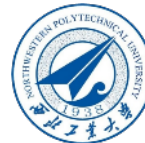


流程控制



- ◆ 掌握布尔表达式
- ◆ 掌握条件表达式
- ◆ 掌握while循环
- ◆ 掌握for循环
- ◆ 掌握循环中的break, continue和else语句



Python程序中的语句默认是按照书写顺序依次被执行的，这时我们说这样的语句是顺序结构的。

- 在顺序结构中，各语句是按自上而下的顺序执行的，执行完上一条语句就自动执行下一条语句，语句之间的执行是不作任何判断的，无条件的。
- 有时候我们需要根据特定的情况，有选择地执行某些语句，这时我们就需要一种选择结构的语句。
- 有时候我们还可以在给定条件下重复执行某些语句，这时我们称这些语句是循环结构的。
- 有了**顺序**、**选择**和**循环**这三种基本的结构，就能够构建任意复杂的程序了。

选择结构和循环结构都会使用布尔表达式作为选择的条件和循环的条件。

- 布尔表达式是由关系运算符和逻辑运算符按一定语法规则组成的式子。
- 关系运算符有：<（小于）、<=（小于等于）、==（等于）、>（大于）、>=（大于等于）、!=（不等于）。
- 逻辑运算符有：and、or、not。
- 布尔表达式的值只有两个，True和False。
- 在Python中False、None、0、“”、()、[]、{}作为布尔表达式的时候，会被解释器看作假（False）。其它的一切都被解释为真，包括特殊值True。

➤ 一个布尔变量可以代表True或False值中的一个。bool函数可以用来（和list、str以及tuple一样）转换其他值。

```
In [99]: ▶ print(type(True))  
          print(bool('Practice makes perfect.'))  
          print(bool(''))  
          print(bool(4))
```

```
<class 'bool'>  
True  
False  
True
```

➤ 布尔表达式的值为False的情况

False、0(0.0、0j)、None、空列表、空元组、空集合、空字典、空字符串、空range对象或其它空迭代对象，Python解释器均认为等价于False。

➤ 布尔表达式的值为True的情况

布尔表达式的值只要不是False，Python均认为与True等价。

➤ Python不同于其它语言，布尔表达式中不能使用赋值运算符“=”，否则会抛出异常。

选择结构通过判断某些特定条件是否满足来决定下一步执行哪些语句。
Python有多种选择语句类型：

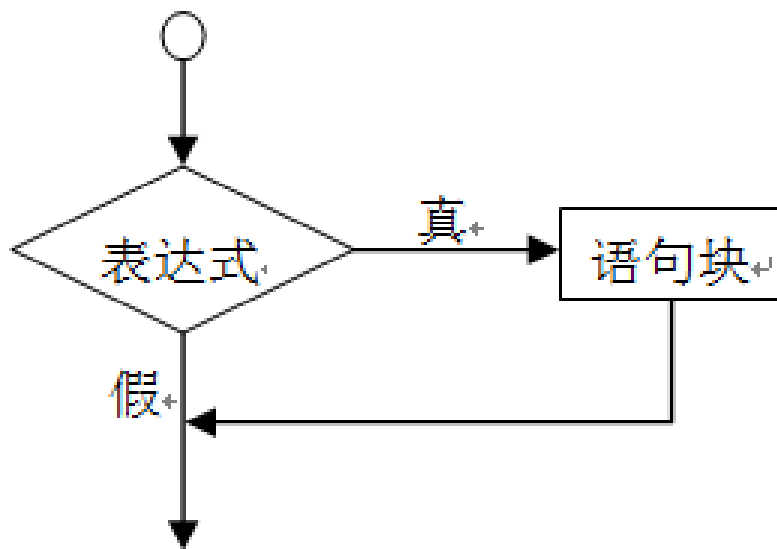
- 单向if语句
- 双向if-else语句
- 嵌套if语句
- 多向if-elif-else语句
- 条件表达式

单向if语句的语法形式如下所示：

if 条件表达式：

满足条件时要执行的语句块

注意：同一语句块的所有语句缩进相同。



- if语句的**语句块**只有当表达式的值为真，即非零时，才会执行；否则的话，程序就会直接跳过这个**语句块**，去执行紧跟在这个**语句块**之后的语句。
- 这里的**语句块**，既可以包含多条语句，也可以只有一条语句。当**语句块**由多条语句组成时，要有统一的缩进形式，相对于if向右至少缩进一个Tab键，否则就会出现逻辑错误，即语法检查没错，但是结果却非预期。

单向if语句例子:

```
In [100]: ▶ math = 65  
print('开始进入if语句并判断表达式的值是否为True')  
if math >= 85:           #条件表达式  
    print('数学成绩优秀') #if语句块, 满足条件执行, 否则不执行  
print('if语句运行结束')  #if语句结构外语句
```

开始进入if语句并判断表达式的值是否为True
if语句运行结束

● 选择结构



➤ **if-else**语句是一种双选结构，根据表达式是真还是假来决定执行哪些语句，它选择的不是做与不做的问题，而是在两种备选操作中选择哪一个操作的问题。

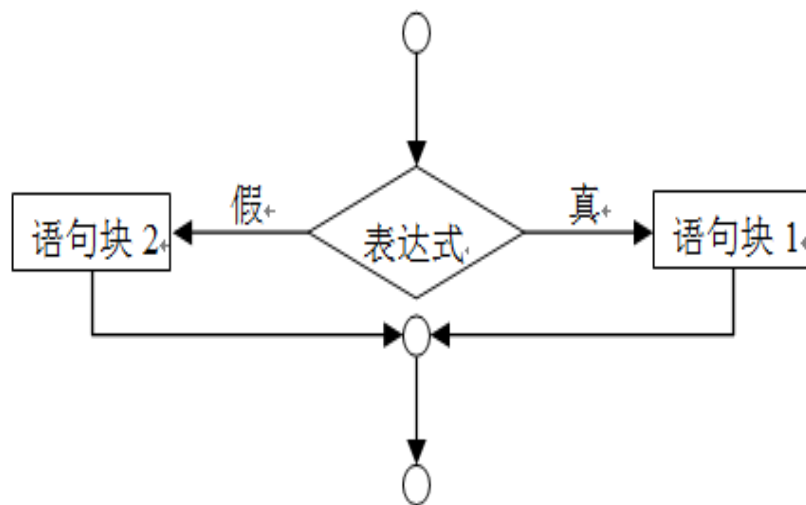
if-else语句的语法形式：

if 布尔表达式：

语句块1

else:

语句块2



从流程示意图中可以看出：当表达式为真（即表达式的值为非零）时，执行语句块1；当表达式为假（即表达式的值为零）时，执行语句块2。

双分支选择结构if-else语句例子：

```
In [104]: ▶ day = "正月初一"
if day == "年三十":           #if语句条件表达式
    print('今天是除夕')       #if语句块, 满足条件执行
else:                          #if语句块, 不满足条件执行
    print('过年了')
    print('可以拿压岁钱了')
```

过年了
可以拿压岁钱了

- 将一个if语句放在另一个if语句中就形成了一个**嵌套if**语句。
- 如果需要在多组操作中选择一组执行，就会用到多选结构，就是**if-elif-else**语句。该语句可以利用**一系列布尔表达式**进行**检查**，并在某个表达式为真的情况下执行相应的代码。
- 虽然if-elif-else语句的备选操作较多，但是**有且只有一组操作**被**执行**，该语句的语法形式如下所示：

if 表达式1 :

 语句块1

elif 表达式2 :

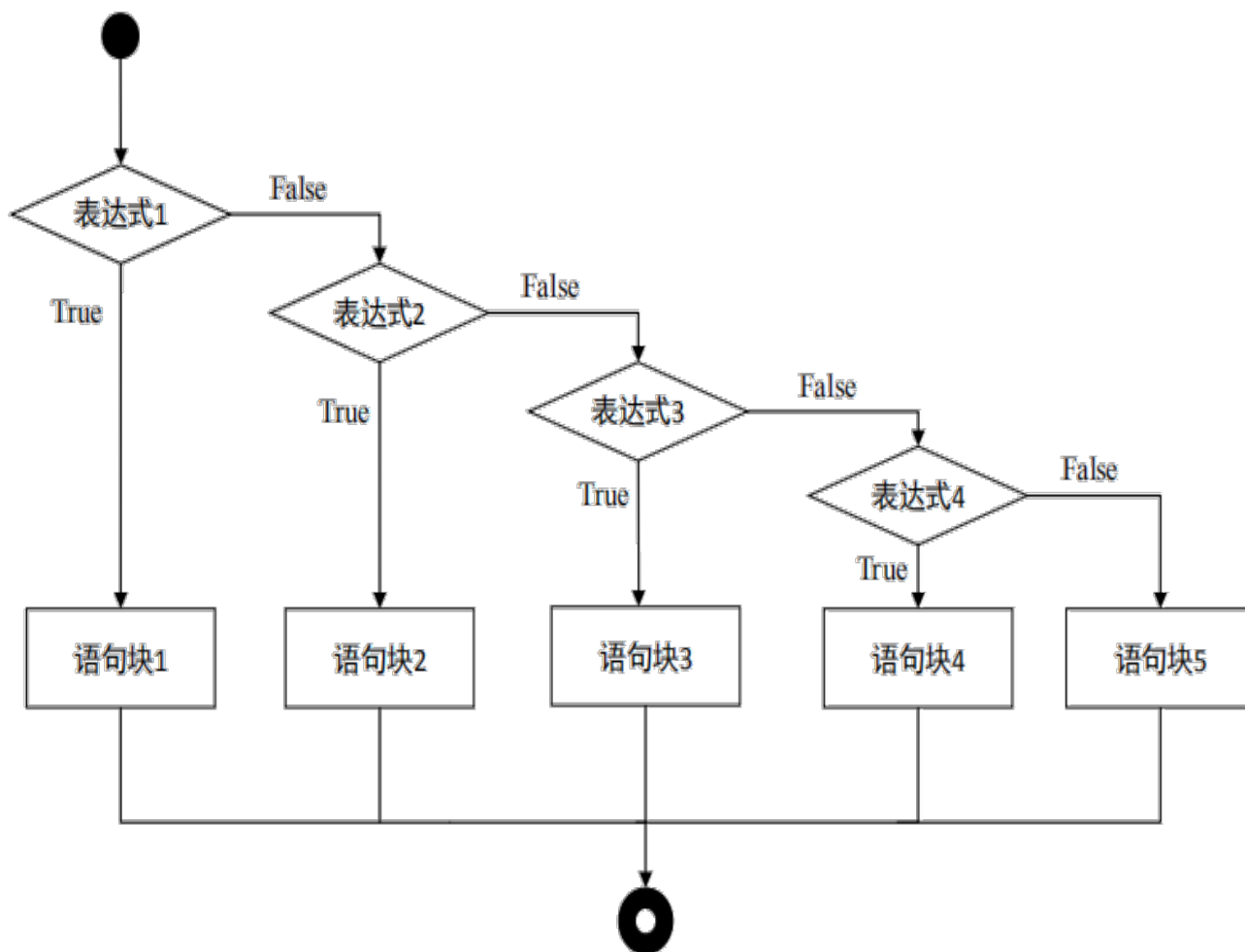
 语句块2

.....

else :

 语句块n

多分支选择结构if-elif-else语句执行流程：



➤ 多分支结构if...elif...else语句例子：

编写程序利用多分支选择结构将成绩从百分制变换到等级制。

```
In [106]: ▶ score=float(input('请输入一个分数：'))
if score>=90.0:
    grade='A'
elif score>=80.0:
    grade='B'
elif score>=70.0:
    grade='C'
elif score>=60.0:
    grade='D'
else:
    grade='F'
print(("祝贺你已经完成成绩分级!你的成绩等级是：")+grade)
```

请输入一个分数：92

祝贺你已经完成成绩分级!你的成绩等级是：A

➤ 给一个变量赋值，但又受一些条件的限制时，可以考虑用**条件表达式**。

➤ 条件表达式的语法结构如下所示：

表达式1 if 布尔表达式 else 表达式2

➤ 如果**布尔表达式为真**，那么这个条件表达式的**结果**就是**表达式1**；**否则**，这个**结果就是表达式2**。

➤ 若想将变量number1和number2中较大的值赋给max，可以使用下面的条件表达式简洁地完成。

max=number1 if number1>number2 else number2

条件表达式示例：

```
▶ number1, number2 = 4.5, 6.8  
max=number1 if number1>number2 else number2  
print("max=", max)
```

max= 6.8

Python循环控制结构：**while**循环和**for**循环

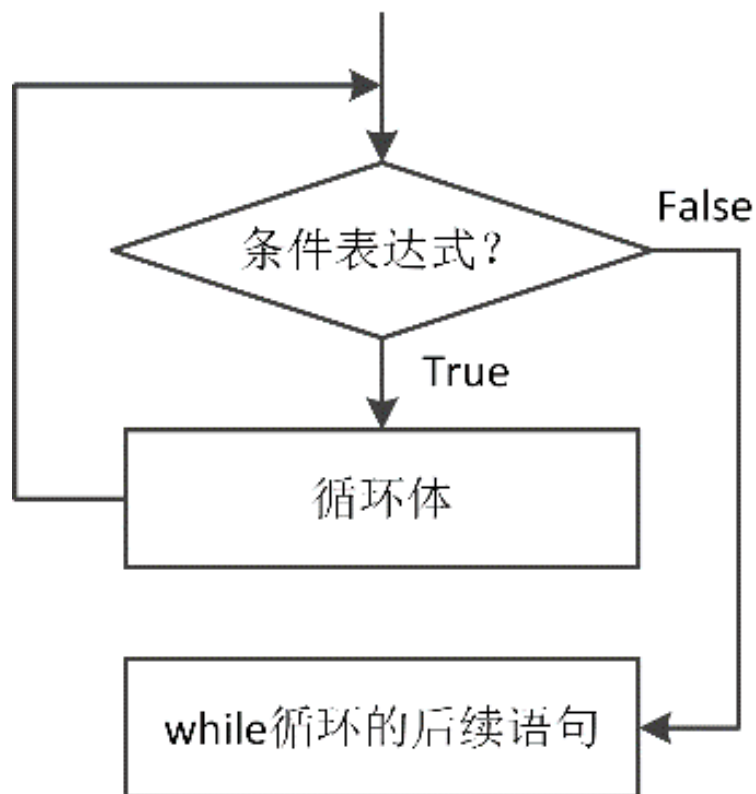
- **while**循环一般用于**循环次数难以提前确定**的情况，也可以用于循环次数确定的情况。
- **for**循环一般用于**循环次数可以提前确定**的情况，尤其是用于**枚举序列或迭代对象中的元素**。

while语句用于在某条件下循环执行某段程序，以处理需要重复处理的任務。其基本形式为：

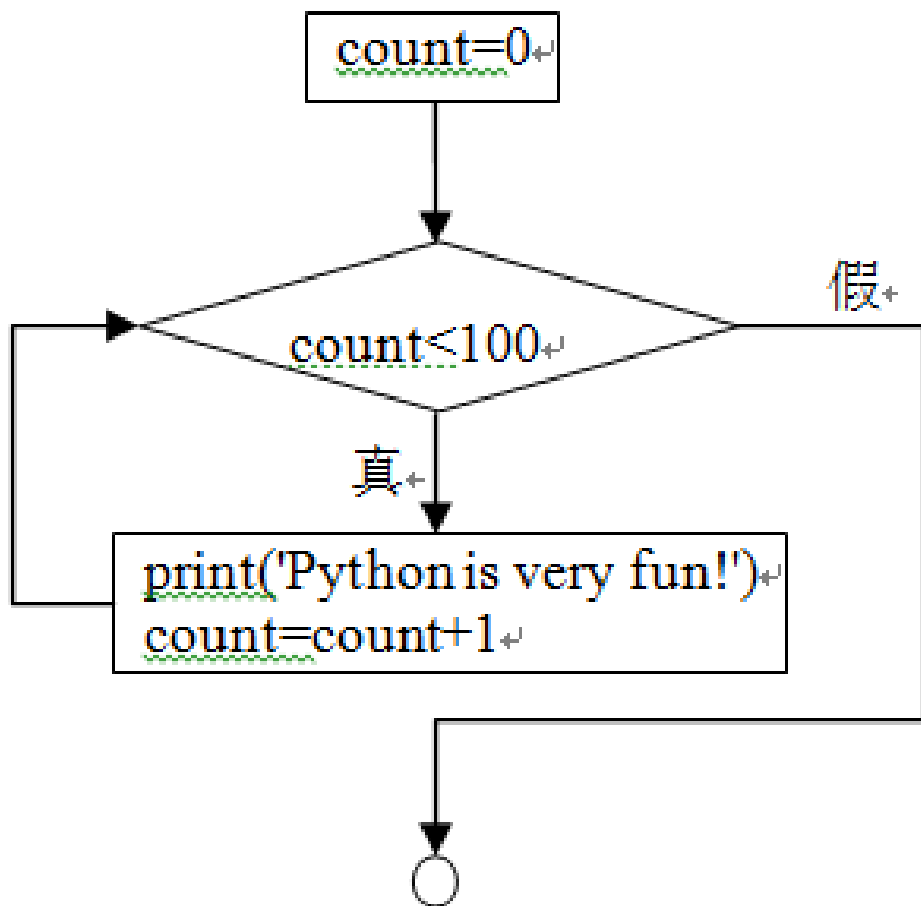
while 循环继续条件：
循环体

- **循环体**可以是一个单一的语句或一组**具有统一缩进的**语句。
- 每个循环都包含一个循环继续条件，即控制循环执行的布尔表达式，每次都计算该布尔表达式的值，如果它的**计算结果为真**，则**执行循环体**；**否则，终止整个循环**并将程序控制权转移到**while循环后的语句**。
- **while循环**是一种**条件控制循环**，它是根据一个条件的真假来控制的。

➤ while循环的执行流程



例1. 显示Python is very fun!一百次的while循环的流程图如下图所示。



注意：要确保循环继续条件最终变成False以便结束循环。编写循环程序时，常见的程序设计错误是循环继续条件总是为True，循环变成无限循环。

例2. 求1~100之间能被5整除，但不能同时被3整除的所有整数。

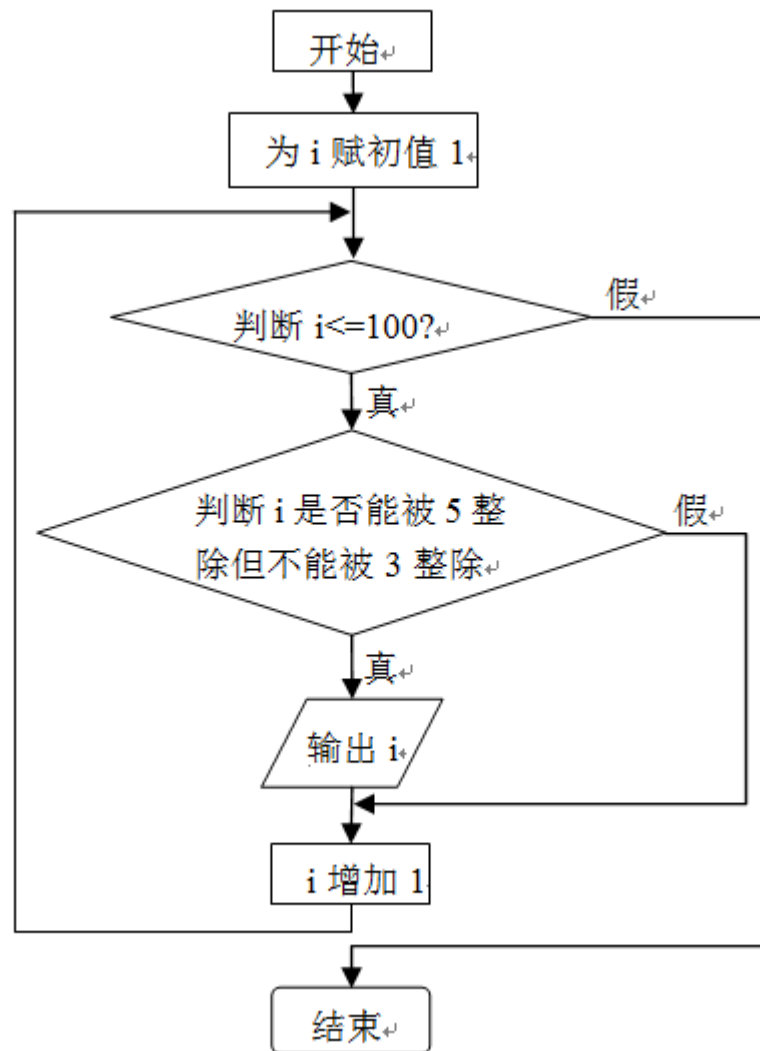
问题分析：

(1) 本题需要对1~100范围内的所有数一一进行判断。

(2) 本题的循环次数是100次。

(3) 在每次循环过程中需要用if语句进行条件判断。

本题整除问题的框图如图右图所示：



例2. 求1~100之间能被5整除，但不能同时被3整除的所有整数。

```
In [107]: ▶ i=1                                #i既是循环变量，同时又是被判断的数
print("1~100之间能被5整除，但不能同时被3整除的所有数是：")
while i<=100:
    if i%5==0 and i%3!=0:    #判断本次的i是否满足条件
        print(i, end=' ')    #打印满足条件的i
    i+=1                    #每次循环i被加1
```

1~100之间能被5整除，但不能同时被3整除的所有数是：

5 10 20 25 35 40 50 55 65 70 80 85 95 100

循环控制策略

要想编写一个能够正确工作的while循环，需要考虑以下三步：

- 第1步：确认需要循环的循环体语句，即确定重复执行的语句序列。
- 第2步：把循环体语句放在循环内。
- 第3步：编写循环继续条件，并添加合适的语句以控制循环能在有限步内结束，即使循环继续条件的值变成**False**。

for循环是一种**遍历型**的**循环**，因为它会依次对某个**序列中全体元素**进行**遍历**，遍历完所有元素之后便**终止循环**。

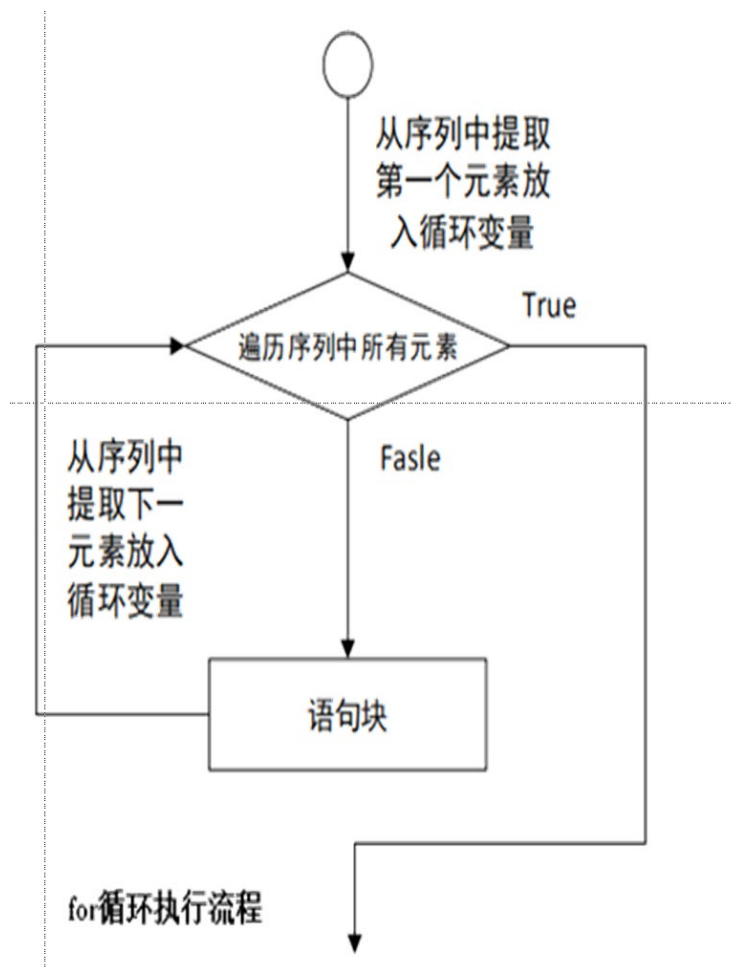
for循环的一般格式如下：

for <循环变量> **in** <可迭代对象或迭代器>：

循环体

- 这里的关键字**in**是**for**循环的组成部分，而非运算符**in**。
- “可遍历序列”里保存了多个元素，且这些元素按照一个接一个的方式存储。
- “**可遍历序列**”被**遍历处理**，每次循环时，都会将“控制变量”设置为“可遍历序列”的当前元素，然后执行循环体。当“**可遍历序列**”中的**元素被遍历一遍**后，即没有元素可供遍历时，**退出循环**。

for语句的示意图：



“遍历序列”可以是列表、元组、字典、集合、文件，甚至可以是自定义类或者函数

for循环可用于**迭代容器**对象中的元素，这些对象可以是**列表、元组、字典、集合、文件**，甚至可以是自定义类或者函数。

(1) 作用于元组

例1. 遍历元组。

```
In [109]: ► test_tuple = [("a",1), ("b",2), ("c",3), ("d",4)]  
print("准备遍历的元组列表:", test_tuple)  
print('遍历列表中的每一个元组:', end='')  
for i, j in test_tuple:  
    print(i, j, end=' ', '')
```

准备遍历的元组列表: [('a', 1), ('b', 2), ('c', 3), ('d', 4)]
遍历列表中的每一个元组:a 1, b 2, c 3, d 4,

(2) 作用于字典

例2. 遍历输出字典元素。

```
In [111]: ▶ person={'姓名':'李明', '年龄':'26', '籍贯':'北京'}  
# item()方法把字典中每对key和value组成一个元组, 并把这些元组放在列表中返回。  
for key,value in person.items():  
    print('key=',key, ', value=',value)  
#只有一个控制变量时, 返回的是每一对key,value对应的元组  
for x in person.items():  
    print(x)  
for x in person:           #不使用items(), 只能取得每一对元素的key值  
    print(x)
```

```
key= 姓名 , value= 李明  
key= 年龄 , value= 26  
key= 籍贯 , value= 北京  
( '姓名', '李明' )  
( '年龄', '26' )  
( '籍贯', '北京' )  
姓名  
年龄  
籍贯
```

(3) 作用于集合

例3. 遍历输出集合元素。

```
In [112]: ► weekdays = {'MON', 'TUE', 'WED', 'THU', 'FRI', 'SAT', 'SUN'}  
# for循环在遍历set时, 元素的顺序和list的顺序很可能是不同的  
for d in weekdays:  
    print (d, end=' ')
```

SUN MON FRI THU TUE SAT WED

(4) 作用于文件

例4. for循环遍历文件，打印文件的每一行。

文件1.txt存有两行文字，在电脑上的路径为： **C:\\python\\1.txt** ：

向晚意不适，驱车登古原。

夕阳无限好，只是近黄昏。

for循环遍历文件，打印文件的每一行。

```
In [26]: ► fd = open('c:\\Python\\1.txt', encoding='utf-8')  
          for line in fd:  
              print(line, end='')  
          fd.close()
```

向晚意不适，驱车登古原。
夕阳无限好，只是近黄昏。

➤ else子句常见用法

● for...else语法形式如下：

for <循环变量> **in** <可迭代对象或迭代器>:
 循环体

else:

 代码块

● while...else语法形式如下：

while 条件表达式:
 循环体

else:

 代码块

➤ else子句常见用法

● while...else例子:

```
In [116]: ▶ count = 0                                #初始化循环控制变量
while count < 5:                                     #条件表达式的值为True时，执行循环体
    print (count, " is less than 5")                 #注意缩进要一致
    count += 1                                       #循环控制变量自增，避免死循环
else:
    print (count, " is not less than 5")             #表达式的值为False时执行else语句块
```

0 is less than 5
1 is less than 5
2 is less than 5
3 is less than 5
4 is less than 5
5 is not less than 5

➤ else子句常见用法

- for...else例子：鸡兔同笼问题。假设共有鸡、兔30只，脚90只，求鸡、兔各有多少只？

```
In [117]: ▶ for ji in range(1, 30):  
            if 2 * ji + (30 - ji) * 4 == 90:  
                print('鸡:', ji, '兔:', 30 - ji)
```

鸡: 15 兔: 15

循环结构的优化

- 在编写循环语句时，应该尽量减少循环内部不必要的计算，将与循环变量无关的代码尽可能放到循环体的外面。

循环结构的优化

➤ 优化前的代码:

```
digits = (1, 2, 3, 4)
```

```
result = []
```

```
for i in range(1000):
```

```
    for i in digits:
```

```
        for j in digits:
```

```
            for k in digits:
```

```
                result.append(i*100+j*10+k) #i,j,k各执行1000 * 4 **3次
```

循环结构的优化

➤ 优化后的代码:

```
digits = (1, 2, 3, 4)
```

```
result = []
```

```
for i in range(1000):
```

```
    for i in digits:
```

```
        i = i * 100
```

#i执行 $1000 * 4$ 次

```
        for j in digits:
```

```
            j = j * 10
```

#j执行 $1000 * 4 ** 2$ 次

```
            for k in digits:
```

```
                result.append(i + j + k)
```

#k执行 $1000 * 4 ** 3$ 次



range()函数用来生成整数数字序列，其语法格式如下：

`range(start, stop[, step])`

参数说明：

- **start**: 计数从start 开始。默认是从0开始。例如range(5)等价于range(0, 5);
- **end**: 计数到**end结束**，但**不包括 end**。例如：range(0, 5)是[0, 1, 2, 3, 4]没有5，也就是说range(a,b)函数返回连续整数a、a+1、...、b-2和b-1的序列；
- **step**: 步长，默认为1。例如：range(0, 5)等价于range(0, 5, 1)。

注意：

range(5)的返回值类型是range类型，如果想得到一个列表，使用list(range(5))得到的就是一个列表[0, 1, 2, 3, 4]。如果想得到一个元组，使用tuple(range(5))得到的就是一个元组(0, 1, 2, 3, 4)。



for循环与range()函数的结合使用



软件学院
School of Software

例. 输出斐波那契数列的前 n 项。斐波那契数列以兔子繁殖为例子而引入，故又称为“兔子数列”，指的是这样一个数列：1、1、2、3、5、8、13、21、34、.....，可通过递归的方法定义： $F(1)=1$ ， $F(2)=1$ ， $F(n)=F(n-1)+F(n-2)$ ($n \geq 3$ ， $n \in \mathbb{N}^*$)。

问题分析：从斐波那契数列可以看出，从第三项起每一项的数值都是前两项（可分别称为倒数第二项、倒数第一项）的数值之和，斐波那契数列每增加一项，对下一个新的项来说，刚生成的项为倒数第一项，其前面的项为倒数第二项。



for循环与range()函数的结合使用



软件学院
School of Software

例. 输出斐波那契数列的前n项。斐波那契数列以兔子繁殖为例子而引入，故又称为“兔子数列”，指的是这样一个数列：1、1、2、3、5、8、13、21、34、……，可通过递归的方法定义： $F(1)=1$ ， $F(2)=1$ ， $F(n)=F(n-1)+F(n-2)$ ($n \geq 3$, $n \in \mathbb{N}^*$)。

```
In [118]: ▶ a, b=1, 1
n=int(input('请输入斐波那契数列的项数(>2的整数): '))
print('前%d项斐波那契数列为: '%(n), end='')
print(a, b, end='')
for k in range(3, n+1):
    c=a+b
    print(c, end='')
    a=b
    b=c
```

请输入斐波那契数列的项数(>2的整数): 12

前12项斐波那契数列为: 1 1 2 3 5 8 13 21 34 55 89 144

● break和continue语句



break和**continue**都是循环控制关键字，为循环结构提供额外的控制。

break和**continue**可以与**for**和**while**循环搭配使用。

break和**continue**只能在循环中使用，不能单独使用。

在嵌套循环中，**break**和**continue**只对最近的一层循环起作用。

break语句和continue语句提供了另一种控制循环的方式。

- **break**语句用来**终止**循环语句，即循环条件没有False或者序列还没被完全遍历完，也会停止执行循环语句。如果使用嵌套循环，break语句将停止执行最深层的循环，并开始**执行下一行代码**。
- **continue**语句**终止当前迭代而进行循环的下一一次迭代**。
- 循环语句也可以带有else子句，else子句在序列遍历结束（for语句）或循环条件为假（while语句）时执行，但循环被break终止时不执行。

● break和continue语句



例. 求*i*除以2的余数，如果余数为0结束当次循环，继续执行后续循环；如果余数不为0则打印*i*，并判断如果*i* >= 7就结束整个循环，执行循环后语句。

```
In [128]: ► for i in range(10):  
            if i % 2 == 0:  
                continue  
            print(i, end=",")  
            if i >= 7:  
                break      #由于break会提前跳出循环体，所以else子句不会得以执行  
        else:  
            print("循环结束")
```

1, 3, 5, 7,

● 总结

- 条件语句**if**可以引入单个分支的条件，也可以结合**elif**或**else**等语句实现多分支的条件。
- **Python**支持**while**循环和**for**循环。
- 应注意**Python**对语句的嵌入式要求，程序块内的多行代码应采用相同的嵌入方式保证各行的语句是对齐的。